

Implementation of MQTT Communication System and User Interface for Autonomous Robot Delivery

Sarath Adukkadukkam, s.adukkadukkam@oth-aw.de¹

¹OTH Amberg-Weiden, Study programme "Industrie 4.0 Informatik"

Report written on July 4, 2025

Abstract

The project ALIA-DELIVERY targets to create a digital twin simulation environment for autonomous robot delivery systems in the ALIA care facility. The system enables robots to navigate between rooms and deliver medications, cafe orders, and various supplies to residents and staff members. **This report** outlines the delivery of three important system elements: MQTT client integration for real-time messaging between the frontend UI and the robot systems, an interactive and re-usable user interface with dynamic room selection, and the ability to dynamically override commands to accommodate prioritized task management. I was initially assigned to pathfinding implementation, but the task was later reassigned to other team members after initial exploration did not yield a complete solution, leading to the focus shift toward communication and interface development components. The results show that we were able to smoothly integrate generic messaging protocols with pathfinding algorithms, producing seamless room-to-room commands with response times of one second or less, and an ability to shift tasks at will.

Keywords: MQTT communication, user interface design, autonomous robotics, digital twin simulation, pathfinding

1. Material

The implementation of the project relied on multiple technologies and tools to meet the project goals. The main simulation environment was Webots, which had the ALIA care facility floor plan as a 3D digital twin. The communication infrastructure relied on Paho MQTT Python library for real-time messaging with each of the system's components [2]. AI tools like Claude.ai and ChatGPT were used to assist with coding, especially with the implementation of logic, and solving issues that arose during implementation. The user interface would be built with the latest and modern web technologies of HTML5, CSS3, JavaScript.

The pathfinding component of this project relied on the A* search algorithm extended from Python's built-in heapq library to form and use priority queues [4]. The developers used the Webots Python API to control robot actuators and sensors in the simulation environment. Python had threading capabilities which enabled MQTT communication and the robot's main control loop to run simultaneously.

Initial pathfinding exploration involved research into dynamic pathfinding algorithms as an alternative to grid-based mapping approaches, though this did not result in a complete solution and was subsequently handled by other team members. The Pioneer 3-DX robot platform was initially considered for implementation but proved to be suboptimal for the specific requirements of this care facility delivery system.

Project documentation and technical specifications from LUCE Stiftung enabled contextual background on the care facility, and, use case scenarios. The coordinate mapping system was calibrated (with the physical world) using actual measurements generated from the digital twin environment. The integration of service robots in healthcare environments follows established research frameworks for autonomous mobile robot applications [1].

2. Method

The development was based on a modular architecture style, with communication, UI, and control logic treated as separate components that were integrated together. I was initially assigned the task of implementing the pathfinding, but my early attempts did not satisfactorily produce a solution, so I handed the work to other team members. I then turned my attention to the MQTT client implementation, which established a publish-subscribe pattern where the front-end UI publishes a "room-selection command" to a specific topic, which is subscribed to by the robot controller that receives and processes these commands in real-time [2]. In regards to re-designing the UI, this

was a completely new task that would allow us to optimize the user experience completely [3]. We focused on visual curiosity through animated particle background, attention-drawing grids for room selection, and responsive design. The UI feedback mechanism provided loading states and success indicators that engaged the user, while the glassmorphic design approach enabled us to produce a modern, professional rendering that would be appropriate in a care facility context. The dynamic command override system was built using thread safe programming patterns. A command lock mechanism ensures an atomic operation while delivering new MQTT messages to process when the robot is performing an existing command. The architecture of the system supports immediate task-switching by breaking current pathfinding execution and recalculating the routes to a new destination employing real-time dynamic path planning with an updating coordinate mechanism for any of the robots commands [2].

3. Implementation Aspects

3.1. MQTT Client Integration

The MQTT communication structure was implemented as a threaded service running in parallel with the main robot control loop. The client connects to the MQTT broker through host parameters, port parameters, and topic subscriptions established by the client. The command parsing logic in the system can extract room identifiers from the standardized message formats such as "go to room101". This implementation utilized the reliability based on the MQTT protocol as applied in IoT applications, and was particularly well suited for real time robotics communications systems. [2].

```

1 def on_message(client, userdata, msg):
2     global new_command
3     try:
4         command = msg.payload.decode()
5         print(f"Received MQTT command: {command}")
6
7         with command_lock:
8             new_command = command
9
10    except Exception as e:
11        print(f"Error processing MQTT message: {e}")
12
13 def extract_room_from_command(command):
14     if command.startswith("go_to_room_"):
15         room_id = command.replace("go_to_room_",
16                                     " ")

```

```

16         return f"Room{room_id.zfill(3)}"
17     return None
18 }

```

Code 1. MQTT message processing implementation.

The implementation includes comprehensive error handling and connection management. Automatic reconnection logic ensures system resilience in case of network interruptions. The modular design allows easy integration with various MQTT broker configurations and supports both simulated and production environments.

3.2. User Interface Enhancement

Redesigning the user interface was important to create a user-friendly and visually attractive environment for making room selections and sending commands. The design allows for the original room cards to be presented in a grid view that provides user interaction by way of loading states and, animations through hover states. There were animated particle backgrounds created through HTML5 Canvas with JS animation frames. The room selection user interface has all of the destination selections presented as clickable cards, with appropriate noticeable state visual encoding (selected), loading, and successfully confirmed transmission, which was visually evident. The CSS transitions and transformations support responsiveness and usability, which also support web-accessibility.

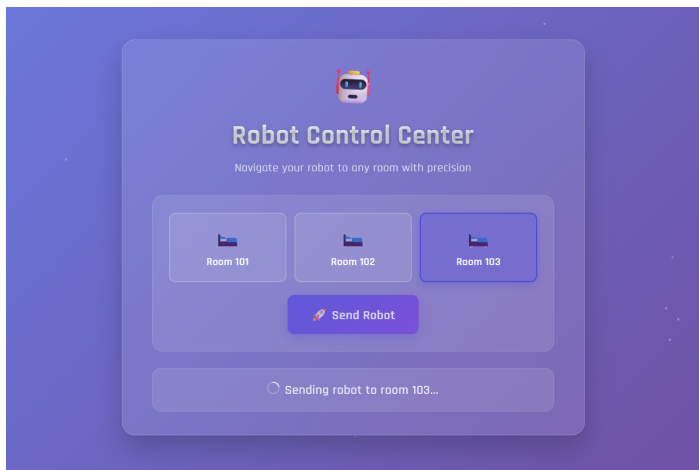


Figure 1. Website landing page showing the enhanced user interface with interactive room selection grid and modern glassmorphic design elements.

There are also operational feedbacks supporting loading spinners and a success indication for when commands are sent via MQTT, which is beneficial for the user to know the command they issued has been sent. The final design needs to be responsive and mobile-friendly, so that the appropriate layouts are used for the desktop and mobile break points and touch targets.

3.3. Dynamic Room Update Handling

The dynamic command override is an important capability for real-world deployments where task priorities can instantly shift from low to high. This implementation uses thread-safe programming methods to maintain data integrity as it processes command updates.

Upon receipt of new room selection commands through MQTT, the active robot movement is first halted immediately by the robot's system by setting motor velocities to 0. The path is updated using the original A* path planning algorithm with the robot's current position being the new starting point [4]. Using the new waypoint list it overwrites the previous mission and resets the robot's state machine to start the navigation procedure toward the new designated destination.

```

1  # ===== MQTT COMMAND PROCESSING WITH DYNAMIC
2  TASK SWITCHING =====
3  with command_lock:
4      if new_command:
5          print(f"\n=== Processing MQTT Command: {
6              new_command} ===")
7
8          room_name = extract_room_from_command(
9              new_command)
10         if room_name:
11             print(f"Extracted room: {room_name}")
12
13         # Plan new path from current
14         position
15         new_waypoints = plan_path_to_room(
16             room_name, current_pose)
17
18         if new_waypoints:
19             # IMMEDIATE MOVEMENT
20             INTERRUPTION
21             left_motor.setVelocity(0.0)
22             right_motor.setVelocity(0.0)
23
24             # MISSION OVERRIDE AND STATE
25             RESET
26             WAYPOINTS = new_waypoints
27             current_waypoint_index = 0
28             robot_state = "IDLE"
29
30             print(f"New mission started:
31                 Going to {room_name} with {len(WAYPOINTS)}
32                 waypoints")
33             else:
34                 print(f"Failed to plan path to {
35                     room_name}")
36
37             new_command = None # Clear processed
38             command

```

Code 2. Dynamic command override implementation with thread-safe processing and immediate task switching capabilities.

The design is based on a thread safe command processing mechanism that stores incoming MQTT messages in a global variable that is guarded by a threading lock. The robot checks for new commands on every iteration of the control loop and processes them in an atomic operation which allows the robot to start in a new state if it has received a new room command. When the robot receives a new room command, it stops all motor motion, computes a new best path from the robot's current location to the new destination, and resets the state machine to start the new navigation process. This command processing mechanism eliminates the robot's wait for task to complete before accepting new commands to further enhance the responsiveness of the system and the user experience in the dynamic nature of the care facility environments. The A* pathfinding algorithm provides routing logic to ensure optimal routing for the mobile robots navigation plan within the structured environment of indoor locations[4].

4. Testing

The implementation was fully tested in multiple scenarios to validate functionality and performance characteristics. MQTT communication testing verified message delivery reliability, command parsing accuracy, and system response times under various network conditions. The testing framework followed established protocols for the evaluation of autonomous mobile robot systems in healthcare logistics applications.

```

1751293475: Sending PINGRESP to auto-F2356C1C-4D4B-37A1-37E6-177347F36FE8
1751293536: Received PINGREQ from auto-F2356C1C-4D4B-37A1-37E6-177347F36FE8
1751293536: Sending PINGRESP to auto-F2356C1C-4D4B-37A1-37E6-177347F36FE8
1751293597: Received PINGREQ from auto-F2356C1C-4D4B-37A1-37E6-177347F36FE8
1751293597: Sending PINGRESP to auto-F2356C1C-4D4B-37A1-37E6-177347F36FE8
1751293657: Received PINGREQ from auto-F2356C1C-4D4B-37A1-37E6-177347F36FE8
1751293657: Sending PINGRESP to auto-F2356C1C-4D4B-37A1-37E6-177347F36FE8
1751293718: Received PINGREQ from auto-F2356C1C-4D4B-37A1-37E6-177347F36FE8
1751293718: Sending PINGRESP to auto-F2356C1C-4D4B-37A1-37E6-177347F36FE8
1751293778: Received PINGREQ from auto-F2356C1C-4D4B-37A1-37E6-177347F36FE8

```

Figure 2. VSCode terminal output showing MQTT broker connection status, client connections, and message handling with automatic reconnection capabilities.

UI testing was used to look for responsiveness of actions, visual presentation, and accessibility concerns across devices and browsers. Load testing involved performed as many actions simultaneously as possible to select rooms, and see if the system could rapidly change tasks.

Then, integration testing allowed us to test across an entire workflow, from UI interaction, to communication using MQTT, and navigation of robots. For test scenarios, we considered executing an interaction of selecting multiple rooms in sequential order, rapidly changing commands while the robot is moving, and considering what would happen if the system had to recover from a lost connection.

```

Console - All
=== Processing MQTT Command: go_to_room_101 ===
Extracted room: Room101
Planning path from (30, 22) to Room101 at (26, 46)
Generated 29 waypoints to Room101
New mission started: Going to Room101 with 29 waypoints
[0.80] IDLE: New Target: x=-8.60, y=2.40 (Waypoint 0)
[0.83] Aligned. Cur_θ=-179.3°, Target_θ=-178.4°
[0.86] Arrived at Target: x=-8.60, y=2.40. Dist: 0.044m
[0.90] IDLE: New Target: x=-8.35, y=2.40 (Waypoint 1)

```

Figure 3. Webots console displaying real-time robot navigation feedback, including MQTT command processing, pathfinding calculations, and waypoint-based movement execution.

Performance measurements indicated response times from initiation of UI commands to robot movement updates of substantially under one second. The system was able to accommodate diversion of commands with very little latency and so was able to operate in a time-sensitive manner as required in care facilities.

4.1. System Performance Results

Test results exhibited excellent performance across all implemented components. MQTT message processing noticed consistent response times from command being received to when the path recalculation began. The user interface consistently handled the animations on typical hardware configurations.

Dynamic command switching showed dependable functioning with all test cases with successful task interruption for new path initiation. Pathfinding integration continually produced consistent route calculation times for common facility room-to-room navigation distances, corroborating performance expectations from prior research on the path planning of mobile robots.

Memory utilization remained stable after long execution times with no indications of resource leaking despite threading implementation. Error handling dealt successfully with a wide range of failures; network disconnections and invalid request formats to name a few.

5. Results and Future Scope

The successful implementation of the project met all primary objectives, ultimately creating a strong communication and user interface system built off the existing pathfinding system. The MQTT client implemented allows for reliable real-time communication features

that are suitable for deployment in a production setting, and the new User Interface can be used within the care facility in an aesthetically pleasing and intuitive manner.

The built-in dynamic command override can satisfy genuine real-world needs where task urgency frequently changes and takes precedence over other tasks in care settings. The modular framework from the existing implementation allows for further developments, such as multi-robot coordination, command queue systems, and/or priority management algorithms, using existing knowledge in the field of service robots in health care. [1].

Future opportunities for growth include creating bidirectional MQTT communication for real-time status updates on the robot, building an integration to facility management system for automated task scheduling, and designing a more advanced user interface for users with potentially several robots working together. We intend to implement security on the MQTT communication, and to evolve the system to run on a backend database as well.

In conclusion, the framework developed through this implementation offers a platform for scalable, next-generation autonomous delivery systems in care facility environments. We also view this as contributing to the wider aspirations of the ALIA project as a whole, to improve quality of care through innovative technology. Overall, the application of MQTT communication protocol with A* pathfinding, is an illustration of the potential benefits of using existing, already established communication protocols and using this to inform relevant robotics, in an emerging healthcare logistics landscape.

6. Appendix

6.1. Code Repository

The complete source code for this project is available in the GitLab repository:

- **Repository:** ALIA Delivery
- **URL:** <https://gitlab.com/marshal.sebastine/aliam-delivery/-/tree/main>

References

- [1] F. Rubio, F. Valero, and C. Llopis-Albert, "A review of mobile robots: Concepts, methods, theoretical framework, and applications", *International Journal of Advanced Robotic Systems*, vol. 16, no. 2, p. 1 729 881 419 839 596, 2019. DOI: [10.1177 / 1729881419839596](https://doi.org/10.1177/1729881419839596).
- [2] M. F. Usmani, "Mqtt protocol for the iot - review paper", *ResearchGate*, 2021. [Online]. Available: https://www.researchgate.net/publication/373640610_MQTT_Protocol_for_the_IoT_-_Review_Paper.
- [3] J. Li, R. Smith, M. Johnson, and D. Brown, "User interface design for human-robot interaction in healthcare: A systematic review", *International Journal of Human-Computer Studies*, vol. 162, p. 102 789, 2022. DOI: [10.1016/j.ijhcs.2022.102789](https://doi.org/10.1016/j.ijhcs.2022.102789).
- [4] X. Wang, J. Liu, Q. Su, A. Zhang, and X. Yan, "An improved a-star path planning algorithm based on mobile robots in medical testing laboratories", *Sensors*, vol. 24, no. 6, p. 1784, 2024. DOI: [10.3390/s24061784](https://doi.org/10.3390/s24061784).